



2

14

1320 Sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Exception handling:
Extra content

عمل طلابي



RB Informatics; 14/11/2024

برمجة



مرحباً بزملائنا المبرمجين!

يسرنا أن نقدم لكم محاضرات الـ Extra، والتي ستتضمن أفكاراً أو أمثلة إضافية على محاضرات النظري تساعد في إثراء المعرفة المتعلقة بموضوع المحاضرة. وننوه إلى أنه عمل طلابي بحث يُدرس بعد المحاضرات الأساسية ولا يغني عنها أبداً. نرجو لكم ولنا التوفيق والمعدلات العالية.

Always remember, you have within you the strength, the patience, and the passion to reach for the stars to change the world!

ما هو الـ (Exception)؟

هو خطأ يحدث أثناء تنفيذ البرنامج ويؤدي إلى تعطيل سيره الطبيعي لأنه لا يؤدي بالضرورة إلى توقف البرنامج بالكامل. يمكننا معالجة الـ exception بشكل يسمح للبرنامج بمواصلة العمل بعد ذلك (ستتعلم معالجتها لاحقاً) في جافا، يتم تمثيل Exceptions من خلال **Exception Class**، مما يسمح لنا بالتعامل مع حالات مثل القسمة على صفر أو القراءة من ملف غير موجود بطريقة بسيطة و شبه تلقائية.

ما هو الـ (Error)؟

يمثل مشاكل تتعلق بالنظام أو الـ **JVM**، وتكون عادةً مشاكل لا يمكن للبرنامج معالجتها مباشرةً، و يستحيل استكمال عمل البرنامج بعد حدوثها وذلك يعني توقف البرنامج المحتوم، ويصعب إصلاحها إلا بتعديل بنية البرنامج أو النظام نفسه. **مثال:** حدوث استهلاك للذاكرة بشكل كبير مما يسبب خطأ يُعرف بـ **StackOverflowError**.

```
1 public static void print(String myString) {
2     print(myString); }
```

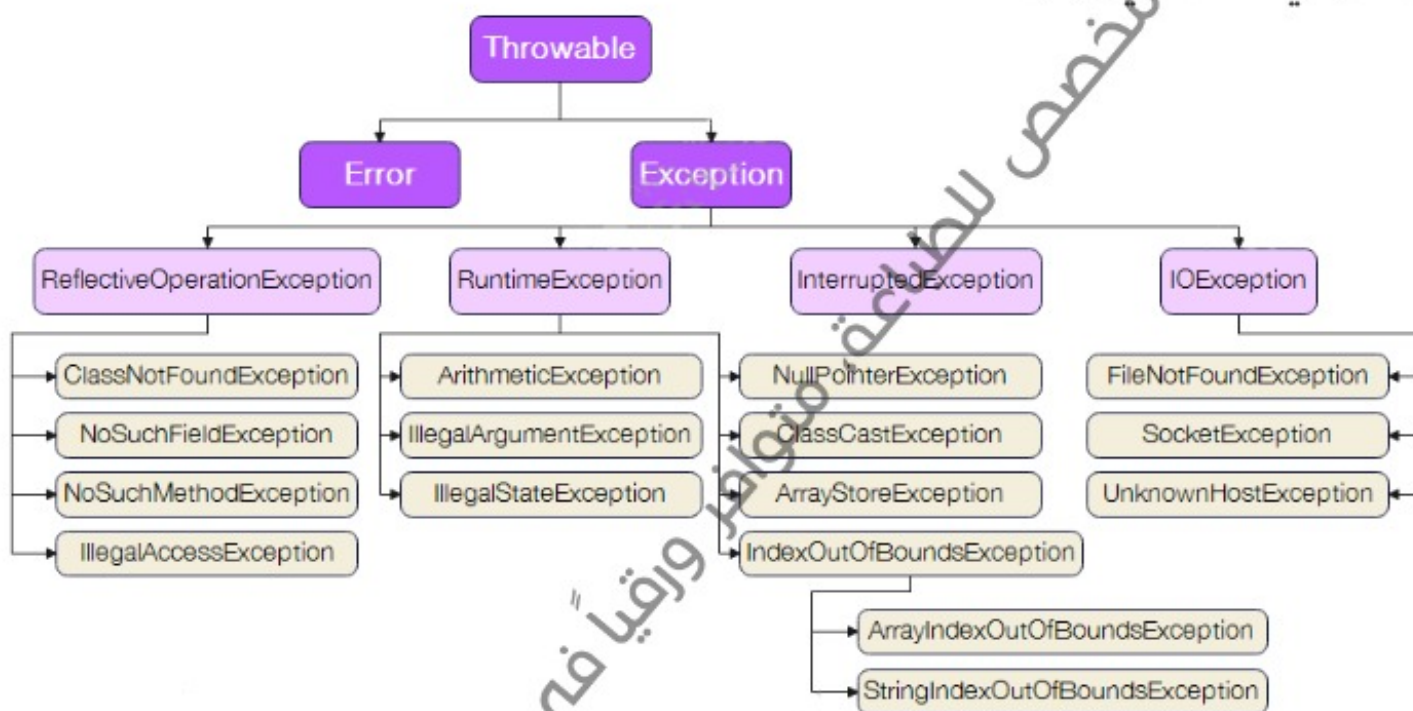
هنا يحدث (recursion) لا نهائي، لأن التابع يستدعي نفسه دون توقف مما يؤدي إلى استنفاد الذاكرة وإيقاف البرنامج نهائياً، و لا يمكن حل هذه المشكلة سوى بتعديل بنية البرنامج ككل و إيقاف التكرار اللانهائي. في جافا يتم تمثيل Errors من خلال **Error Class**

JVM (Java Virtual Machine):

بيئة تشغيل افتراضية للغة جافا تستدعي تابع main في الكود، وتعمل كوسيط بين البرنامج المكتوب ونظام التشغيل حيث تجعل لغة جافا مستقلة عن الآلة.

Exception Hierarchy

في جافا، كل من Error و Exception يرثان من Class يسمى Throwable وهو الـ Class الذي يمثل جميع أنواع الأخطاء التي قد تحدث في برنامجك.



Types of Exceptions

تُصنّف هذه الـ exceptions بشكل عام إلى:

User-Defined Exception

Built-in Exception

Unchecked Exception

لا يمكن اكتشافها إلا في حال حدوثها في الـ run-time ، ويفضل دوماً محاولة التعامل معها لتجنب توقف البرنامج بشكل مفاجئ.

Checked Exception

يمكن للـ Compiler توقع الجزء من الكود الذي يمكن أن يحدث فيه الـ Exception أثناء مرحلة الـ Compiling ، و بالتالي يجبرك الـ Compiler على التعامل معه و لن يعمل البرنامج حتى تقوم بالتعامل معه فعلياً.

يوفر كلاس Throwable عدة طرق لمعرفة تفاصيل الخطأ مثل:

- `getStackTrace()` للحصول على تسلسل الأحداث قبل حدوث الخطأ مما يسهل تتبع أصل المشكلة .
- `toString()` تظهر اسم الخطأ الذي حدث و تفصيل بسيط.

معالجة الـ Exceptions:

عند حدوث exception داخل method تقوم هذه الـ method بإنشاء object حسب نوعه يحتوي على اسم الـ exception ووصفه وحالة البرنامج عند حدوثه، وهذا ما يدعى بالـ `Throwing`.

تبحث الـ JVM عن تعليمات تعالج الـ exception في الـ call stack، وهذه التعليمات تدعى بالـ `Exception handler`، حيث يبدأ البحث عنها في الـ method نفسها، وفي حالة عدم وجوده يتم البحث في الـ methods الأخرى ضمن الـ call stack بشكل معاكس لترتيب استدعائها، وهذا ما يُعرف بالـ `Exception Propagation`.

إن بحثت الـ JVM في جميع الـ methods ضمن الـ call stack ولم تجد handler مناسباً، عندها تسلمه الـ handler الافتراضي الذي يقوم بطباعة معلومات الـ exception وإنهاء البرنامج.

```
function one(){
  two();
}

function two() {
  three();
}

function three() {
  console.trace("Call Stack");
}
```

Call Stack

ما الـ `Call stack`؟

لو أردنا وضع 3 كتب بشكل عرضي على مكتب ما، سنضع الكتاب الأول، فوقه الكتاب الثاني، فوقه الكتاب الثالث.

هنا في حال أردنا استعادة هذه الكتب، سنأخذ الكتاب الثالث، يليه الكتاب الثاني، ويليه الكتاب الأول (بعكس ترتيب وضعها)، وهذا ما يُدعى بمبدأ الـ `(Last In First Out) LIFO`.

الـ `methods` هي `calls` تتخزن في بنية تتعامل بهذا المبدأ، وهي الـ `call stack`، حيث:

حين يتم استدعاء `method 1`، تتم إضافة `frame` جديد إلى الـ `call stack` يحتوي على اسم الـ `method` والـ `parameters` والـ `local variables` الخاصة بها وعنوان الـ `return` الذي يجب أن يعود إليه البرنامج عند الانتهاء من تنفيذها، وإذا قامت الـ `method 1` خلال تنفيذها باستدعاء `method 2`، ستم إضافة الـ `method 2` إلى الـ `call stack` بنفس الطريقة، وإذا انتهى تنفيذ الـ `method 2`، تتم إزالة الـ `frame` الخاص بها من الـ `call stack` والعودة للـ `method 1` التي استدعتها، وحين يتم الانتهاء من تنفيذ الـ `method 1` وإخراجها من الـ `call stack` وإفراغه من الـ `methods` سينتهي تنفيذ البرنامج.

حسناً الـ exceptions هي أخطاء يمكن معالجتها و التصرف بطريقة معينة عند مواجهتها لإكمال سير البرنامج بشكل طبيعي.

و نتعامل مع هذا برمجياً من خلال التالي:

throws - finally - throw - catch - try

1. try: تحدد المكان الذي قد يحدث فيه الخطأ.

2. catch: تحدد الطريقة التي سنعالج فيها exception إن حدث ضمن try.

3. finally: تحدد كود سيعتقد في أي حالة سواء حدث exception أم لم يحدث.
(نستخدمها لتحديد أمر ضروري الحدوث بأي حال "كإغلاق الـ file")

4. throw: تجعل الكود يصدر exception يدوياً من النقطة التي نكتبها فيها.

5. throws: تجعل التابع الذي توضع ضمنه يتوقع أن يصدر exception ضمنه مما يجعل التابع المنادي لهذا التابع يتعامل مع الـ exception بدوره.

بتفصيل بسيط:

نحيط الجزء من الكود الذي نشك أنه قد يقع به exception مثل القسمة على صفر أو فتح فايل غير موجود بـ **try** block.

بعد هذا نضع **catch block** التي بدورها تستقبل باراميتراً من نوع exception حيث ستقوم باستقبال الـ exception الذي قد يصدر عن الـ try في حالة تماثل البارميتر الخاص بها و الـ exception الحاصل.

- يمكن جعل catch block واحدة تعالج خطأ أو أكثر من نوع من الأخطاء .
- و يمكن عمل أكثر من catch block كل واحد منها يعالج خطأ و عند صدور أحد الأخطاء يتم معالجته بالـ catch block المناسبة.
- في حال عدم حدوث أي خطأ لا يتم تنفيذ أي منها و يستكمل تنفيذ الكود بالشكل الطبيعي.

finally

وظيفتها تكمن في أنه عند انتهاء تنفيذ ما نجريه ضمن try و عند انتهاء تنفيذ أحد الـ catch في حال حدث exception سيتم تنفيذها مهما يكن و تستخدم لإغلاق الملفات عادةً.

ملاحظة: عند استخدام try لا بد على الأقل من استخدام catch block وحيدة أو finally ويمكن استخدام كليهما سوياً.

أما throw

فهي keyword تسمح لنا بإصدار exception من النقطة التي نكتبها فيها ضمن الكود و هي نفس العملية التي تحدث تلقائياً عند حدوث exception ضمن try .
فعندما تحدث القسمة على صفر مثلاً فإن java تعرف أنه exception فتقوم باستخدام throw تلقائياً من try block و هذا ما يجعل catch block قادرة على تلقي الخطأ و التعامل معه .

ملاحظة :

بلغات البرمجة ك cpp التي تكون أقل ديناميكية من جافا في runtime فإنه كنا نضطر لاستخدام throw يدوياً عند استخدام try و catch ، بحيث نضع شرطاً مثلاً و عند تحققه سنقوم بعمل throw لexception ما تقوم catch بالتقاطه بعدها.

throws

هي keyword توضع في توقيع التابع مع نوع exception لتشير إلى أن هذا التابع قد يصدر منه exception من نوع ما و على المبرمج لهذا التابع أن يتعامل معه في حال لزم ذلك.
نلاحظ أثر هذا عند الـ checked exception لأنه سيصبح المبرمج مجبراً على التعامل مع هذا الـ exception عند مناداة هذا التابع، في مثال لاحق سنوضح هذا.

throw	throws
تُستخدم في block الدالة بعدها instance للـ exception الملقى.	تُستخدم في توقيع الدالة بعدها اسم الـ class للـ exception الملقى.
تلقى exception واحد.	يمكن استخدامها إلقاء عدة exceptions.



كل ما سيتلو هذه النقطة عبارة عن أمثلة و تمارين مع بعض الملاحظات:

مثال 1:

```
1 public class Main {
2     public static void main(String[] args) {
3
4     try {
5         //1 يطبع
6         System.out.println(1);
7         // يحدث exception من نوع Arithmetic Exception
8         System.out.println(1/0);
9         // هنا بشكل غير صريح يتم القيام بالتالي
10        //throw new ArithmeticException();
11
12        // exception لن يتم تنفيذ هذا السطر بعد حدوث ال
13        System.out.println(2);
14    }
15    // بما ان نوع ال exception الذي تم عمل throw له من
16    // try block مماثل لما لدينا كبارميتر هنا سيتم تنفيذ هذه
17    // catch block
18    catch (ArithmeticException e)
19    {
20        //3 يطبع
21        System.out.println(3);
22    }
23    finally {
24        // بعد انتهاء السابق يطبع 4
25        System.out.println(4);
26    }
27 }
```

غير مخصص للطباعة، متوافر وارقياً في المكتبة العمومية



```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7
8     try {
9         // ندخل رقمين نريد ناتج قسمة x/y حيث y هو المقسوم عليه
10        // من المحتمل أن يحدث exception من نوع InputMismatchException في حال أدخل المستخدم قيمة ليست int
11        int x = scanner.nextInt();
12        int y = scanner.nextInt();
13        // يطبع 1 في حال عدم حدوث exception
14        System.out.println(1);
15        // هنا من المحتمل أن يحدث exception بناءً على قيمة y فلو كانت 0 يحدث ArithmeticException
16        System.out.println(x/y);
17        // يتم طباعة 2 لو لم يحدث exception
18        System.out.println(2);
19    }
20    // لدينا 2 catch blocks لو حدث قسمة على 0 لن ننفذ الأولى لو حدث خلل بالإدخال ننفذ الثانية
21    catch (ArithmeticException e)
22    {
23        // يطبع 3 عند تنفيذها
24        System.out.println(3);
25    }
26    catch (InputMismatchException e){
27        // يطبع 6 عند تنفيذها
28        System.out.println(6);
29    }
30    finally {
31        // بعد انتهاء السابق يطبع 4
32        System.out.println(4);
33    } }

```

Output:

في حال حدوث إدخال من نوع خاطئ:	في حال حدوث قسمة على 0:
6	1
4	3
	4
في حال استكمال البرنامج دون خلل وإدخال x=10, y=2 (مثلاً):	
1	
5	
2	
4	

```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7         // إدخال x و y حيث y هنا هي المقسوم عليه
8         int x = scanner.nextInt();
9         int y = scanner.nextInt();
10
11     try {
12         // إذا كان المقسوم عليه 0 قم برمي exception من نوع arithmetic برسالة خاصة
13         if(y == 0)
14         {
15             // نقوم بإصدار exception عندها سيتوقف البرنامج ليبحث عن catch block توافق الـ exception الذي تم إصداره
16             throw new ArithmeticException("Y CAN'T BE 0" /*CUSTOM MESSAGE YOU CAN CHOOSE*/);
17         }
18         // ليس من الضروري التقييد بذلك فحسب يمكنك ابتكار شروطك الخاصة و استخدام الـ throw كما يحلو لك
19         if(x<0)
20         {
21             // نقوم بإصدار exception عندها سيتوقف البرنامج ليبحث عن catch block توافق الـ exception المرمي
22             throw new ArithmeticException("x Can't be negative");
23         }
24         System.out.println(x/y);
25
26     }
27
28     catch (ArithmeticException e)
29     {
30         System.out.println(e);
31     }
32
33
34     finally {
35
36         System.out.println(4);
37     } }

```

Output:

في حال إدخال $y=0$:

```
java.lang.ArithmeticException: Y CAN'T BE 0
4
```

في حال إدخال $x<0$:

```
java.lang.ArithmeticException: x Can't be negative
4
```

في حال استكمال البرنامج دون خلل وإدخال $y=2, x=10$ (مثلاً):

```
5
4
```

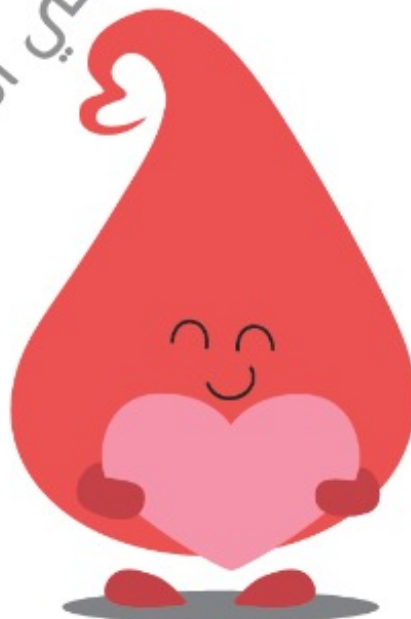
1. عند طباعة e ضمن catch block فيتم استدعاء toString و طباعة اسم الخطأ.
2. يمكن كتابة catch block واحدة تعالج أكثر من نوع exception على النحو التالي:

```
catch (ArithmeticException | InputMismatchException e)
```

```
{
    System.out.println(e);
```

```
}
```

3. يمكن كتابة عدد كبير من ال exceptions دفعة واحدة و تفصل بينهم باستخدام |
4. يمكنك عمل exception خاص بك حيث تقوم بعمل class ، إن أردت عمله على أنه checked اجعله يرث من Exception ، و إن أردت جعله unchecked اجعله يرث من RuntimeException .
5. لو تركنا throw ل exception دون التعامل معها سيتوقف عمل البرنامج و كأنما حدثت المشكلة بشكل طبيعي دون أن نقوم بذلك.
لهذا لا نقوم نحن بإصدار exception باستخدام throw إلا في حالات معينة خاصة أن جافا قامت بجعل هذه العملية شبه أوتوماتيكية للأخطاء الشائعة.



غير منصوص للطباعة متوافق ورقياً في المكتبة المعتمدة

```

1. class Main
2. {
3.     String str = "a";
4.     void A()
5.     {
6.         try
7.         { str += "b";
8.           B();
9.         }
10.        catch (Exception e)
11.        { str += "c"; }
12.    }
13.    void B() throws Exception
14.    {
15.        try
16.        { str += "d";
17.          C();
18.        }
19.        catch(Exception e)
20.        { throw new Exception(); }
21.        finally
22.        {
23.            str += "e";
24.        }
25.        str += "f";
26.    }
27.    void C() throws Exception
28.    {
29.        throw new Exception();
30.    }
31.    void display()
32.    {
33.        System.out.println(str);
34.    }
35.    public static void main(String[] args)
36.    {
37.        Main object = new Main();
38.        object.A();
39.        object.display();
40.    }
41. }

```

- في هذا المثال، يتم إنشاء object من كلاس Main وتعريف متحول str من نوع String وإعطائه القيمة الابتدائية a.
- يتم استدعاء الدالة A()، وفيها يُضاف المحرف b إلى str ثم استدعاء الدالة B().
- نلاحظ وضع تعليمة استدعاء B() (السطر 8) في try block بعدها catch block تستقبل بارميتر من نوع Exception، مما يعني أنه في حال إلقاء exception من B() دون معالجة ستتم معالجته في A().
- في الدالة B() يُضاف المحرف d إلى str ثم استدعاء الدالة C()، ونلاحظ في (السطر 17) نفس الملاحظة السابقة.
- الدالة C() عند استدعائها تقوم برمي exception.
- تلتقط catch block في B() exception لتعالجه فتقوم بإعادة رميه.
- يتم رمي exception خارج الدالة B() بعد عدم إيجاد catch block أخرى تعالجه.
- يتم تنفيذ finally وإضافة المحرف e إلى str.
- لن يتم تنفيذ السطر 25 بسبب رمي exception.
- تلتقط catch block في A() exception لتعالجه فتقوم بإضافة المحرف c إلى str.
- في هذه النقطة تم الانتهاء من تنفيذ الدالة A() والعودة للدالة main.
- يتم استدعاء الدالة display التي ستقوم بطباعة str بقيمتها الحالية (abdec).

Output:
abdec

```

1 import java.io.BufferedWriter;
2 import java.io.File;
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.util.InputMismatchException;
6
7 public class Main {
8     public static void main(String[] args) {
9
10         System.out.println("a");
11         // استدعاء تابع
12         hi();
13         System.out.println("d");
14
15     }
16     public static void hi()
17     {
18         System.out.println("b");
19         try {
20             FileWriter fileWriter = new FileWriter("/TEST");
21         }
22         catch (IOException E){
23         }
24         System.out.println("c");
25     }
26 }

```

هذا checked exception يدعى IOException يعطى بمحاضرة أخرى، لكن ما يهم الآن أنه من نوع الأخطاء الذي يجبرنا المترجم على التعامل معه، هنا في الحل الأول تم إجبارنا على التعامل مع الـ FileWriter لأنه يحتمل أن يحدث FileNotFoundException ، الآن في الوضع الحالي تعاملنا معه بشكل طبيعي فأحطناه بـ try-catch .

الحل الآخر أن نعالجه بالتابع الذي يستدعيه. لنجبر التابع الذي يستدعيه على ذلك نستخدم throws keyword ضمن توقيع التابع، وبالتالي سيصبح في حالتنا main هي التابع الذي سيتعامل مع الـ Exception.

```

1 import java.io.BufferedWriter;
2 import java.io.File;
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.util.InputMismatchException;
6
7 public class Main {
8     public static void main(String[] args) {
9         System.out.println("a");
10        // استدعاء تابع
11        try {
12            hi();
13        }
14        catch (IOException E){
15        }
16
17        System.out.println("d");
18    }
19    public static void hi() throws IOException
20    {
21        System.out.println("b");
22        FileWriter fileWriter = new
23        FileWriter("/TEST");
24        System.out.println("c");
25    } }

```

ملاحظة 1: حتى لو استخدمنا try و catch ضمن التابع نفسه الآن سيبقى هنالك compile error ، لأنك أخبرت المترجم بأنك ستعالج الخطأ في التابع المستدعي.

ملاحظة 2: يمكن جعل main method تقوم بعمل throws كذلك و بالتالي لن تعود مضطراً لمعالجة الـ exception بـ try و catch ، لكن في حال حدوث exception سيتوقف البرنامج عن العمل.

```

1 import java.io.BufferedWriter;
2 import java.io.File;
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.util.InputMismatchException;
6
7 public class Main {
8     public static void main(String[] args) {
9         System.out.println("a");
10        try
11        {
12            hi1();
13        }
14        catch (IOException | InterruptedException E)
15        {
16            System.out.println(E);
17        }
18    }
19    public static void hi1() throws IOException,
InterruptedException {
20        System.out.println("b");
21        hi2();
22        System.out.println("e");
23        FileWriter fileWriter = new FileWriter("/TEST");
24        System.out.println("f");
25    }
26    public static void hi2() throws InterruptedException {
27        System.out.println("c");
28        Thread.sleep(1000);
29        System.out.println("d");
30    }
31 }

```

هنا في التابع hi2 لدينا exception قد يحدث بسبب اعتراض الـ thread أثناء التعامل مع resource بيانات مشترك مع threads أخرى بغض النظر إنه checked exception فها بدلاً من التعامل معه على مستوى التابع هذا جعلنا التابع الذي يستدعيه يتعامل معه و هو hi1 الذي بدوره فيه المشكلة السابقة ذاتها IOException، فالآن يتوجب على التابع hi1 التعامل مع مشكلته و مشكلة التابع hi2، بعدها قمنا برمي مشكلتي التابعين hi1 و hi2 للمنهج main حيث أصبحت مسؤوليته علاج مشكلتي التابعين كونه مستدع لهما، عالجنا المشكلة هناك باستخدام try and catch .

المشكلة التي ستحدث هنا هي الـ FileNotFoundException

سيطبع هنا:

```

a
b
c
d
e
java.io.FileNotFoundException: \TEST (تم رفض الوصول)

```

مثال 7 (مهم):

```

1
2 public class Main {
3     public static void main(String[] args) {
4         int y = 1;
5
6         System.out.println(hi(y));
7     }
8     public static int hi(int y) {
9         try {
10             System.out.println("hello");
11             int t = 2/y;
12             return t;
13         }
14         catch (ArithmeticException e)
15         {
16             return 0;
17         }
18         finally
19         {
20             return 1;
21         }
22     }
23 }

```

في هذه الحالة تحديداً يحدث سلوك غريب و هو أن return statement الخاصة بـ finally ستقوم بعمل override لجميع الـ return statements في catch و try ما يجعلها تنفذ دائماً بغض النظر عن الحالة، فهنا سواء حدث exception أم لم يحدث قيمة ما سيرجعه التابع ستكون 1؛

Output:

```
hello
1
```

فعندما يكون لدينا finally فيها return statement ستكون النقطة المرجعية التي منها سيقوم التابع بإرجاع قيمة مهما حدث.

هنا بغض النظر حدث exception أم لا ستقوم بإغلاق الـ reader ضمن finally للحفاظ على الموارد بكافة أشكالها سواء من ذاكرة أو threads أو file handles لأنه عندما يحدث خلل أثناء استخدام الـ reader or the writer فأحياناً لن يتم ضمن البرنامج تحرير الموارد التي تم تخصيصها للتعامل مع file ما، لهذا ضمن finally نغلقه مما يؤدي لتحرير الموارد و الحفاظ على أداء البرنامج.

```

1. import java.io.BufferedReader;
2. import java.io.FileNotFoundException;
3. import java.io.FileReader;
4. import java.io.IOException;
5. public class Main {
6.     public static void main(String[] args) {
7.         BufferedReader reader = null;
8.         try {
9.             reader = new BufferedReader(new
10.                 FileReader("/test/file"));
11.         }
12.         catch (FileNotFoundException e){
13.             System.out.println(e);
14.         }
15.         finally {
16.             try {
17.                 reader.close();
18.             }
19.             catch (IOException e) {
20.                 System.out.println(e);
21.             }
22.         }
23.     }
24. }

```

نستعرض القليل من الـ exceptions الشائعة:

IllegalArgumentException:

يحدث عندما تتلقى الدالة بارميتر لا يتناسب مع شرط معطى داخلها (يتعلق بمدخلات الدالة).

IllegalArgumentException:

يحدث عندما يتم استدعاء دالة في وضع غير مناسب لعملها (يتعلق بحالة الكائن عند استدعاء الدالة).

```
1 public class Main {
2     public static void main(String[] args) {
3         // null pointer exception
4         String x = null;
5         try {
6             System.out.println(x.length());
7         }
8         catch (NullPointerException e) {
9             System.out.println(e);
10        }
11        //ArrayIndexOutOfBoundsException
12        int [] arr = new int[5];
13        try {
14            System.out.println(arr[10]);
15        }
16        catch (ArrayIndexOutOfBoundsException e) {
17            System.out.println(e);
18        }
19        //NumberFormatException
20        String invalidNumber = "abc";
21        try {
22            int number =
Integer.parseInt(invalidNumber);
23        }
24        catch (NumberFormatException e) {
25            System.out.println(e);
26        }
27        //IndexOutOfBoundsException
28        ArrayList<String> list = new ArrayList<>();
29        try {
30            list.get(1);
31            list.remove(0);
32        }
33        catch (IndexOutOfBoundsException e){
34            System.out.println(e);
35        }
```

```
36        // IllegalArgumentException
37        try {
38            triggerIllegalArgumentException(-10);
39        }
40        catch (IllegalArgumentException e)
41        {
42            System.out.println(e);
43        }
44        // IllegalStateException
45        int n1=7;
46        int n2=-6;
47        if(n1>=0 && n2>=0)
48        {
49            addition(n1,n2);
50        }
51        else
52        {
53            throw new IllegalStateException("Either
one or two numbers are not Positive Integer");
54        }
55        }
56        public static void
triggerIllegalArgumentException(int age) {
57            if (age < 0) {
58                throw new
IllegalArgumentException("Age cannot be
negative");
59            }
60        }
61        public static void addition(int a,int b)
62        { System.out.println("Addition of Positive
Integers :"+(a+b));
63        }
64 }
```



- انتهت المحاضرة -